Ark — Sprint 1: Single User Production (Production Spec)

Goal: An initial single-user production tool where an artist creates a project, defines scenes/shots, generates AI images/videos per shot, manages references and elements, and uses an AI chat assistant. Built as a collaborative-ready base — all decisions are made with the explicit intent of adding full multi-user collaborative features in Sprint 2, without refactoring.

Sprint 1 = single user, collaboration ready. One user, one workspace initially. All data models, APIs, and database design are built from day one to support Sprint 2 collaborative features (global board, canvas/whiteboard, Kitsu integration, multi-user, real-time sync, audio generation, comments) with zero schema migrations or breaking changes. Nothing is built as a sandbox throwaway — everything is production grade.

Architecture

```
┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│  Frontend    │  REST    │  Backend     │          │ Object Storage│
│  (Next.js)   │◄────────►│  (Django)    │◄────────►│  Bucket      │
│              │  JSON    │              │          │              │
└──────────────┘          └──────────────┘          └──────────────┘
```

- **Next.js frontend** — all UI, no server-side logic. Calls the Django API for everything.
- **Django backend** — REST API. Handles auth, project CRUD, file management, generation job dispatch, object storage operations.
- **Object Storage** — stores all files. Images, videos, project manifests, generation outputs.

Hosting & infrastructure decisions deferred: Deployment platform, cloud provider, and hosting specifics will be decided at a later date. This spec is completely provider-agnostic.

Sprint 1 operation: Frontend polls backend API for job status updates. In Sprint 2, real-time WebSocket support will be added.

No additional external services in Sprint 1. Generation jobs are handled directly by the Django backend process queue.

Database

Production Database: PostgreSQL 16. Production grade from day 1. No SQLite, no throwaway prototype database.

- All models, indexes, and constraints are defined exactly as they will run in production
- Recursive CTE support used for version history tree queries, element traversal, and asset lineage
- JSONB column type used for all unstructured data:
  - `model_settings`
  - `prompt_tags`
  - `cinematographer_settings`
  - `hexcodes`
- All foreign keys are properly indexed
- Full transaction safety on all operations

No additional external services in Sprint 1. Simple in-memory queue for generation jobs.

Prompt Tags System

Prompt tags are pre-defined modifier values that are:

- Stored as separate structured JSON fields in the database
- Shown as separate UI controls in the generation pane
- Automatically appended or merged with the user's prompt when sending to the model
- **Never stored inline within the prompt text**

Prompt tags are separate from model parameters.

Prompt tag types:

Tag type	Description
Cinematographer	Camera, lighting, and style settings. Shown as 3-tab modal. Automatically appended as natural language description to the end of the prompt.
Hexcodes	Color palette presets. Up to 5 hex color values stored as an array. Rendered as swatches in the UI. Injected as a color guide into the prompt.
Reference tags	`@img1`, `@Element1` references. Resolved to actual assets at generation time.

Model Settings (separate, not prompt tags)

Model settings are **not prompt tags**. They are passed directly as native parameters to the model API, and are never injected into the prompt text. These are:

- CFG scale
- Steps
- Seed
- Resolution
- Aspect ratio
- Motion strength
- All other model-native parameters

Model settings are stored separately as `model_settings` JSON field. They are never merged into the prompt.

Object Storage Folder Structure

Object Storage is a flat file store. Every file lives in exactly ONE location on Object Storage — where it was originally created or uploaded. Files are **never copied** between shots. Instead, the database tracks role assignments (output, input, pinned) that link files to shots.

```
{project_id}/
    ├── elements/                           # All project elements (characters, locations, props)
    │     └── {element_id}/
    │           ├── frontal.png             # Primary reference image
    │           ├── ref_01.png              # Additional reference angles (optional)
    │           ├── angle_sheet.png         # Auto-generated angle sheet fallback
    │           └── meta.json               # Name, description, category, tags
    │
    ├── generations/                        # ALL generated files (flat)
    │     ├── 1711892400_ayush_kling_a3f2.mp4
    │     ├── 1711892500_ayush_flux_b7c1.png
    │     └── ...
    │
    └── manifest.json                       # Project state + metadata
```

Core principle: Tag, don't copy

A file's physical location on Object Storage never changes after creation. When an artist uses SH02's output as a reference in SH05, we **don't** copy the file. Instead, we create a `FileAssignment` record in the database:

```
FileAssignment(file=file_xyz, shot=sh05, role="input", assigned_by=ayush)
```

The same file can have many assignments:

- `file_xyz` → output of SH02 (where it was generated)
- `file_xyz` → input ref of SH05 (where it's being used as reference)
- `file_xyz` → input ref of SH08 (reused again)
- `file_xyz` → pinned by user (available as quick-access across all shots)

Soft-delete: hide, never destroy

Files are **never physically deleted** from Object Storage until the project is archived. Instead:

- A `deleted_at` timestamp is set on the File record.
- All normal queries filter `WHERE deleted_at IS NULL`.
- Deleted files disappear from the grid, board, and all views.
- A **Trash view** shows all soft-deleted files with the option to restore.
- Because files are never actually removed, no links or lineage chains ever break.

Why this approach?

Old (copy-based)	New (tag-based)
Copying a ref duplicates the file on Object Storage	Zero duplication — one file, many tags
Deleting a source breaks copies' provenance	Nothing ever breaks — file always exists
Each shot folder is self-contained but bloated	Shots are lightweight — just assignment records
Object Storage storage grows with every ref usage	Object Storage only grows with new generations/uploads
Hard to track "where is this file used?"	Query all assignments for any file instantly

Data Model

manifest.json

Lives on Object Storage. The Django backend reads/writes it. Serves as a portable snapshot of the project state.

```
{
  "id": "proj_abc123",
  "name": "Scarecrow",
  "created_by": "ayush",
  "created_at": "2026-03-01T00:00:00Z",

  "scenes": [
    {
      "id": "sc01",
      "name": "SC01",
      "order": 0,
      "shots": [
        {
          "id": "sh01",
          "name": "SH01",
          "order": 0,
          "direction_note": "Wide establishing shot, cornfield at dawn",
          "tags": ["exterior", "dawn", "wide"],
          "published_file_id": "file_abc123"
        }
      ]
    }
  ],

  "files": {
    "file_abc123": {
      "s3_key": "generations/1711892400_ayush_kling_a3f2.mp4",
      "origin": "generation",
      "user": "ayush",
      "model": "kling-1.6",
      "prompt": "A weathered scarecrow walks through golden corn...",
      "prompt_tags": ["scarecrow", "cornfield", "dawn", "cinematic"],
      "cinematographer": {
        "camera_body": "ARRI Alexa 65",
        "lens": "Cooke S7/i",
        "focal_length": "35mm",
        "f_stop": "f/1.4",
        "lighting": "Golden Hour",
        "style": "Photorealistic"
      },
      "model_settings": {
        "aspect_ratio": "16:9",
        "duration": 5,
        "resolution": "1080p",
        "seed": 42,
        "guidance": 7.5
      },
      "refs_used": ["file_def456"],
      "elements_used": [
```

```
        { "name": "Element1", "element_id": "elem_scarecrow_01" }
      ],
      "created_at": "2026-03-31T14:00:00Z",
      "deleted_at": null,
      "width": 1920,
      "height": 1080,
      "duration": 5.0,
      "extension": "mp4",
      "is_video": true
    }
  },

  "assignments": [
    {
      "file_id": "file_abc123",
      "shot_id": "sh01",
      "role": "output",
      "assigned_by": "ayush",
      "assigned_at": "2026-03-31T14:00:00Z"
    },
    {
      "file_id": "file_abc123",
      "shot_id": "sh05",
      "role": "input",
      "assigned_by": "sarah",
      "assigned_at": "2026-03-31T15:20:00Z"
    },
    {
      "file_id": "file_abc123",
      "shot_id": null,
      "role": "pinned",
      "assigned_by": "ayush",
      "assigned_at": "2026-03-31T14:05:00Z"
    }
  ]
}
```


Key differences from a copy-based model:

- `files` contains each file exactly once, keyed by file ID, with its Object Storage location and generation metadata.
- `assignments` is the join table — it maps files to shots with a role. One file can appear in many assignments.
- `refs_used` on a file now references other file IDs (not Object Storage paths), since files are never copied.
- `deleted_at` is tracked per file — `null` means active, a timestamp means soft-deleted.

Django models

The key insight: **File** stores what a file IS (metadata, Object Storage location). **FileAssignment** stores where a file is USED (which shot, what role). One file → many assignments.

```
class Project(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    name = models.CharField(max_length=255)
    s3_prefix = models.CharField(max_length=255)  # "{project_id}/"
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    # Sprint 2: kitsu_id = models.CharField(max_length=100, blank=True, null=True, unique=True)

class Sequence(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    project = models.ForeignKey(Project, related_name="sequences", on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    order = models.IntegerField(default=0)
    tags = models.JSONField(default=list, blank=True)  # ["exterior", "night"]
    # Sprint 2: kitsu_id

class Shot(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    sequence = models.ForeignKey(Sequence, related_name="shots", on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    order = models.IntegerField(default=0)
    direction_note = models.TextField(blank=True, default="")
    tags = models.JSONField(default=list, blank=True)  # ["wide", "establishing"]
    published_file = models.ForeignKey("File", null=True, blank=True,
                                       on_delete=models.SET_NULL, related_name="+")
    # Sprint 2: kitsu_id

class File(models.Model):
    """A single file on Object Storage. Never duplicated, never physically deleted until project arc
    One File can be assigned to many shots with different roles."""
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    project = models.ForeignKey(Project, related_name="files", on_delete=models.CASCADE)
    s3_key = models.CharField(max_length=500, unique=True)
    origin = models.CharField(max_length=20)
      # "generation" — AI-generated output
      # "upload" — user-uploaded reference or asset
      # "extraction" — extracted frame from video
      # "edit" — created in Image Lab (vary, upscale, inpaint, etc.)

    # Who created this file
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)

    # Soft delete — never physically remove until project archive
    deleted_at = models.DateTimeField(null=True, blank=True)
    deleted_by = models.ForeignKey(User, null=True, blank=True,
                                   on_delete=models.SET_NULL, related_name="+")

    # Generation metadata (populated when origin="generation" or "edit")
```

```
    model_id = models.CharField(max_length=100, blank=True, default="")
    prompt = models.TextField(blank=True, default="")
    prompt_tags = models.JSONField(default=list, blank=True)
    cinematographer_settings = models.JSONField(default=dict, blank=True)
    model_settings = models.JSONField(default=dict, blank=True)
    refs_used = models.ManyToManyField("self", symmetrical=False, blank=True,
                                       related_name="used_as_ref_in")
    elements_used = models.JSONField(default=list, blank=True)

    # File properties
    width = models.IntegerField(default=0)
    height = models.IntegerField(default=0)
    duration = models.FloatField(default=0)
    extension = models.CharField(max_length=10)
    is_video = models.BooleanField(default=False)
    is_audio = models.BooleanField(default=False)
    file_size_bytes = models.BigIntegerField(default=0)

    # Derived from a parent file (e.g. upscale of file X, extracted frame from file Y)
    parent_file = models.ForeignKey("self", null=True, blank=True,
                                    on_delete=models.SET_NULL, related_name="children")

    class Meta:
        indexes = [
            models.Index(fields=["project", "deleted_at"]),
            models.Index(fields=["project", "origin"]),
        ]

    @property
    def is_deleted(self):
        return self.deleted_at is not None

class FileAssignment(models.Model):
    """Links a File to a Shot with a role. This is the 'tag' — one file can have many assignments.
    This replaces copying files between shots."""
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    file = models.ForeignKey(File, related_name="assignments", on_delete=models.CASCADE)
    shot = models.ForeignKey(Shot, related_name="file_assignments",
                             on_delete=models.CASCADE, null=True, blank=True)
      # shot=NULL for project-level assignments (pinned, global assets)
    role = models.CharField(max_length=10)
      # "output" — this file was generated FOR this shot
      # "input"  — this file is used as a reference IN this shot
      # "pinned" — this file is pinned for quick access (shot can be NULL for global pins)
    assigned_by = models.ForeignKey(User, on_delete=models.CASCADE)
    assigned_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = [("file", "shot", "role")]
```

```
        indexes = [
            models.Index(fields=["shot", "role"]),
            models.Index(fields=["file"]),
        ]

class Element(models.Model):
    """Unified element for characters, locations, and props.
    All project assets use the same Element form and structure.
    Frontal image is required. Additional reference images are optional.
    For models that don't support native element APIs (e.g. non-Kling),
    an auto-generated angle sheet is used as a single reference fallback."""
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    project = models.ForeignKey(Project, related_name="elements", on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True, default="")
    category = models.CharField(max_length=50)  # "characters" | "locations" | "props"
    frontal_image = models.ForeignKey(File, on_delete=models.CASCADE, related_name="+")
    reference_images = models.ManyToManyField(File, blank=True, related_name="+")  # optional, up to
    angle_sheet = models.ForeignKey(File, null=True, blank=True,
                                    on_delete=models.SET_NULL, related_name="+")  # auto-generated
    tags = models.JSONField(default=list, blank=True)
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)

class GenerationJob(models.Model):
    """Tracks async generation jobs dispatched to backend worker queue."""
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    shot = models.ForeignKey(Shot, related_name="jobs", on_delete=models.CASCADE)
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    model_id = models.CharField(max_length=100)
    prompt = models.TextField()
    prompt_tags = models.JSONField(default=list, blank=True)
    cinematographer_settings = models.JSONField(default=dict, blank=True)
    model_settings = models.JSONField(default=dict, blank=True)
    ref_files = models.ManyToManyField(File, blank=True, related_name="used_in_jobs")
    element_ids = models.JSONField(default=list, blank=True)
    status = models.CharField(max_length=20, default="pending")
      # "pending" | "processing" | "completed" | "failed"
    progress = models.IntegerField(default=0)  # 0-100
    error_message = models.TextField(blank=True, default="")
    result_file_id = models.CharField(max_length=100, blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

Backend API (Django REST Framework)

Auth

```
POST   /api/auth/login/          # Email + password → JWT token
POST   /api/auth/register/
POST   /api/auth/refresh/
```

Projects

```
GET    /api/projects/                      # List user's projects
POST   /api/projects/                      # Create project (with scenes/shots)
GET    /api/projects/{id}/                 # Get project with scenes + shots
PUT    /api/projects/{id}/                 # Update project name
DELETE /api/projects/{id}/                 # Archive project
```

Scenes

```
GET    /api/projects/{id}/scenes/                  # List scenes
POST   /api/projects/{id}/scenes/                  # Add scene
PUT    /api/projects/{id}/scenes/{scene_id}/       # Rename, reorder, update tags
DELETE /api/projects/{id}/scenes/{scene_id}/
```

Shots

```
GET    /api/scenes/{scene_id}/shots/               # List shots with file counts
POST   /api/scenes/{scene_id}/shots/               # Add shot
PUT    /api/shots/{shot_id}/                        # Update name, direction note, order, tags
DELETE /api/shots/{shot_id}/
POST   /api/shots/{shot_id}/publish/               # Publish a candidate
```

Files

```
GET    /api/projects/{id}/files/                    # List all active files in project
GET    /api/projects/{id}/files/?origin=generation   # Filter by origin
GET    /api/projects/{id}/trash/                    # List soft-deleted files
POST   /api/files/upload/                           # Upload a new file (creates File record)
DELETE /api/files/{file_id}/                         # Soft delete (sets deleted_at)
POST   /api/files/{file_id}/restore/                # Restore from trash (clears deleted_at)
```

File Assignments (the core relationship API)

```
GET    /api/shots/{shot_id}/assignments/            # List all assignments for a shot
GET    /api/shots/{shot_id}/assignments/?role=output # Only outputs
POST   /api/shots/{shot_id}/assignments/            # Tag a file to a shot with a role
DELETE /api/assignments/{assignment_id}/             # Remove a tag (file stays, just unlinked)
```

Elements (Characters, Locations, Props)

```
GET    /api/projects/{id}/elements/                 # List all elements
GET    /api/projects/{id}/elements/?category=characters  # Filter by category
POST   /api/projects/{id}/elements/                 # Create element (frontal + optional refs)
PUT    /api/elements/{element_id}/                  # Update name, description, category, add refs
DELETE /api/elements/{element_id}/
```

Generation

```
POST   /api/generate/                   # Submit generation job
GET    /api/generate/{job_id}/          # Poll job status
GET    /api/generate/?shot={shot_id}    # List jobs for a shot
```

Prompt History (no separate table)

⚠️ **No separate PromptHistory model required.** All prompt + prompt tag metadata already exists on every File record.

When a user clicks:

- ✅ **Reuse Prompt**
- ✅ **Undo / Redo**
- ✅ **Retry**

The frontend simply reads:

```
File record fields:
  prompt                    -> raw user prompt text
  prompt_tags               -> all prompt tags (cinematographer, hexcodes, etc)
  model_settings            -> CFG, steps, seed, resolution
  model_id                  -> which model was used
```

Every generation already stores the full exact state that created it. There is no need to duplicate this data into a separate history table. Recent prompts are just the most recent File records filtered by user.

No additional API endpoints required. All history is already present.

Chat (AI Prompt Assistant)

```
POST   /api/chat/
```

Object Storage Presigned URLs

```
GET    /api/files/{file_id}/url/        # Get presigned download URL (expires 1hr)
POST   /api/files/upload-url/           # Get presigned upload URL for direct-to-Object Storage
```

Feature-by-Feature Implementation

Feature 1: Home Screen (Project Selector)

What it does: The landing page. Shows all projects as cards with thumbnails, metadata, and status. Click one to enter the workspace. Has a "New Project" card.

Reference: `src/components/HomeScreen.tsx`

```
┌──────────────────────────────────────────────────────────────┐
│  [A] Ark                                   [Search] [Avatar]│
├──────────────────────────────────────────────────────────────┤
│  Projects                                                    │
│  Select a project to open the workspace                      │
│                                                              │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐    │
│  │ ▶ video  │  │          │  │          │  │   + New  │    │
│  │ thumbnail│  │  letter  │  │  letter  │  │  Project │    │
│  │          │  │          │  │          │  │          │    │
│  ├──────────┤  ├──────────┤  ├──────────┤  │  (dashed │    │
│  │Scarecrow │  │Moonrise  │  │Deep Cur..│  │  border) │    │
│  │5 scenes  │  │3 scenes  │  │8 scenes  │  │          │    │
│  │47 shots  │  │28 shots  │  │112 shots │  │          │    │
│  │In Prod.  │  │Pre-Prod. │  │In Prod.  │  │          │    │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘    │
└──────────────────────────────────────────────────────────────┘
```

Each project card contains:

- **Thumbnail area** (top ~60%): Either a video thumbnail from the most recent published output (auto-plays on hover, pauses on leave) or a fallback letter avatar with the first character of the project name on a colored background.
- **Info area** (bottom ~40%):
  - Project name (truncated with ellipsis if > 20 chars)
  - `{scene_count} scenes · {shot_count} shots`
  - Status badge — pill-shaped, color-coded:
    - "In Production" → emerald/green
    - "Pre-Production" → amber/yellow
    - "Review" → violet/purple
    - "Complete" → blue
  - "Updated 2h ago" timestamp (relative)
- **Hover state**: card lifts -1px, border turns orange, glow intensifies. Video thumbnail auto-plays.
- **Click**: navigates to `/project/{id}` (workspace).

"New Project" card:

- Dashed border, centered "+" icon and "New Project" text.
- Hover: border turns solid orange.
- Click: opens CreateProjectModal.

Backend:

```
class ProjectListView(generics.ListCreateAPIView):
    serializer_class = ProjectSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Project.objects.filter(
            created_by=self.request.user
        ).annotate(
            scene_count=Count("scenes"),
            shot_count=Count("scenes__shots"),
        ).order_by("-updated_at")
```

Sprint 1 Scope

⚠️ **EXPLICIT SPRINT 1 FOCUS:**

For Sprint 1 we are **only building the Image and Video generation tabs**. All other tabs, modes, and features are deferred to later sprints:

- ✅ **IN SCOPE:** Image tab, Video tab
- ❌ **NOT IN SCOPE:** Audio tab, 3D tab, Special tab, Audio generation, Chat, Canvas, Board, Kitsu integration, multi-user

All features documented in this spec refer exclusively to the Image and Video generation workflows.

Feature 2: Create Project Flow

What it does: Modal where you name a project and define the production hierarchy.

⚠️ **Film optimized structure with Kitsu compatibility:** This matches film production workflow, and maps directly to Kitsu structure without modification. No structural changes required for Sprint 2 Kitsu integration.

```
┌──────────────────────────────────────────┐
│  Create New Project                      │
│                                          │
│  Project Name                            │
│  ┌──────────────────────────────────┐    │
│  │ Scarecrow                        │    │
│  └──────────────────────────────────┘    │
│                                          │
│  Sequences                               │
│  ┌──────────────────────────────────┐    │
│  │ SQ01  Opening         8 shots  [x]  │    │
│  │ SQ02  Chase           12 shots [x]  │    │
│  │ SQ03  Confrontation   6 shots  [x]  │    │
│  │                                  │    │
│  │ [+ Add Sequence]                 │    │
│  └──────────────────────────────────┘    │
│                                          │
│  ── or ──                                │
│  [Import from Kitsu]  (Sprint 2 — grayed)  │
│                                          │
│              [Cancel]  [Create Project]     │
└──────────────────────────────────────────┘
```

Hierarchy (film production):
`Project → Sequence → Shot`

Episodes are intentionally omitted for film workflow. This maps directly to Kitsu's `Project → Sequence → Shot` when importing film projects. For TV projects, episode support will be added as an optional toggle later.

Sequence row fields:

Field	Type	Default	Notes
Name	text input	SQ01, SQ02... auto-increment	Editable
Description	text input	empty	Optional subtitle

Field	Type	Default	Notes
Shot count	number input	8	Min 1, max 200
Delete [x]	button	—	Removes row. Min 1 sequence required.

Buttons:

- **[+ Add Sequence]** — appends a new row with auto-incremented name.
- **[Import from Kitsu]** — disabled/grayed in Sprint 1, shows "Coming in Sprint 2" tooltip. Renders the button so the UI space is reserved.
- **[Cancel]** — closes modal, no changes.
- **[Create Project]** — validates (name required, >= 1 sequence), calls `POST /api/projects/`, on success navigates to workspace.

Backend on create:

1. Creates Project record with object storage prefix
2. Creates Object Storage folder: `{project_id}/assets/characters/`, `assets/locations/`, `assets/props/`, `elements/`
3. For each sequence: creates Sequence record
4. For each shot in each sequence: creates Shot record linked to parent sequence
5. Writes initial `manifest.json` to Object Storage
6. Returns project with nested sequences → shots

Note: The existing `Scene` model is renamed to `Sequence`. All existing code references to Scene will map directly to Sequence. 100% backwards compatible, no logic changes required.

Feature 3: Main Workspace Layout

What it does: The primary working screen after entering a project. Two-panel layout: generation controls on the left, shot grid on the right.

Reference: `src/app/page.tsx`, `src/components/ControlsPane.tsx`, `src/components/FileBrowser.tsx`

```
┌──────┬──────────────────┬──────────────────────────────┐
│ Rail │  Controls Pane   │        Shot Grid             │
│ 48px │     ~350px       │        flex: 1               │
│      │                  │                              │
│ Logo │  Model Selector  │  [<] Scarecrow / [SC03 ▼] 47  │
│      │  References      │                              │
│ ──── │  Elements        │  ┌─ SH05 (active) ──────────┐│
│ Img  │  Prompt Box      │  │ [ref] [ref] [gen] [gen]  ||
│ Vid  │  Cinematographer  │  │ [gen] [gen] [🔄 45%]     ||
│ Tool │  Parameters      │  └──────────────────────────┘│
│ Aud  │  Advanced        │  ┌─ SH06 (dimmed) ──────────┐│
│ Chat │  ──────────      │  │ [ref] [gen] [gen]        ||
│ ──── │  [Generate]       │  └──────────────────────────┘|
│ Hist │                  │  ┌─ SH07 (dimmed) ──────────┐│
│ Set  │                  │  │ [gen] [gen]              ||
│      │                  │  └──────────────────────────┘│
└──────┴──────────────────┴──────────────────────────────┘
```

Three columns:

Column 1: Mode Rail (48px wide)

A narrow vertical strip on the far left.

- **Logo** at top: "A" with orange gradient border.
- **Mode buttons** (36x36px, `rounded-lg`):
  - ✅ **Image** — photo icon. Active: orange text, orange left bar, subtle orange bg. **IN SPRINT 1 SCOPE**
  - ✅ **Video** — camera icon. **IN SPRINT 1 SCOPE**
  - ❌ **Tools** — wrench icon. **DISABLED IN SPRINT 1**
  - ❌ **Audio** — music note icon. **DISABLED IN SPRINT 1**
  - ❌ **Chat** — chat bubble icon. **DISABLED IN SPRINT 1**
- **Divider line** between mode buttons and utility buttons.
- **Utility buttons** at bottom:
  - **History** — clock icon. (Sprint 2: full generation history viewer)
  - **Settings** — gear icon. (Sprint 2: project settings, model API keys)

Active state: orange gradient background (`from-orange-600 to-amber-400`), black icon, glow shadow. Inactive: gray icon, transparent bg, hover brightens.

Disabled during generation: Image/Video/Tools tabs are disabled (opacity-50, cursor-not-allowed) while a generation is in progress.

Column 2: Controls Pane (~350px)

The main interaction panel. Content changes based on the active mode tab. See Features 7-9 for detailed breakdowns of each tab.

Column 3: Shot Grid (flex: 1)

- **Header bar:**
  - Back button [<] — returns to home screen
  - Project name (text, not clickable)
  - Scene selector dropdown — see Feature 5
  - Shot count ("47 shots")
- **Shot containers** stacked vertically — see Feature 4

Feature 4: Shot Container

What it does: A visual container for one shot. Displays the shot's outputs and input refs in a flat grid. Has a header with navigation and a shot picker dropdown.

Reference: `src/components/ShotContainer.tsx`

Shot Header

```
┌──────────────────────────────────────────────────────────┐
│ ▲  [SH05 ▼] — Bedroom confrontation    [👤][👤]  12 cand.  │
│ ▼                                                         │
```

Element	Behavior
▲ / ▼ arrows	Navigate to previous/next shot in the scene. Clicking ▲ on the first shot does nothing.
[SH05 ▼]	Shot name, clickable. Opens a dropdown listing all shots in the scene. Selecting one activates that shot. Dropdown closes on outside click.
— Bedroom confrontation	Direction note. Short text describing the shot intent. Editable on double-click (Sprint 2).
[👤] avatars	Element avatars. If Elements are tagged to this shot, their frontal thumbnails show as 20x20 circles. Sprint 2: also shows active user avatars.
12 candidates	Total count of files (outputs + refs) in this shot.

Candidate Grid

A flat CSS grid, 3 columns, `gap-2`. Refs appear first, then outputs, sorted by `created_at` ascending.

```
┌──────────┐  ┌──────────┐  ┌──────────┐
│ emerald  │  │ emerald  │  │          │
│ border   │  │ border   │  │  gen 1   │
│  (ref)   │  │  (ref)   │  │          │
└──────────┘  └──────────┘  └──────────┘

┌──────────┐  ┌──────────┐  ┌──────────┐
│          │  │  ▶ play  │  │ ★ pub'd  │
│  gen 2   │  │ (video)  │  │  (star)  │
│          │  │          │  │          │
└──────────┘  └──────────┘  └──────────┘

┌──────────┐
│ 🔄 gen   │
│ 45%...   │
└──────────┘
```

Candidate Tile — Complete Button Map

Each tile is 16:9 aspect ratio, `rounded-[14px]`, with layered hover controls.

Border colors by role:

Role	Border	Shadow
`ref` (input)	2px emerald-500	`0 0 12px rgba(16,185,129,0.25)`
`pinned` (global pin)	2px indigo-500	`0 0 12px rgba(99,102,241,0.25)`
`output` (default)	1px transparent → amber-400/30 on hover	none

Always-visible indicators (no hover needed):

Indicator	When	Appearance
★ Published badge	`published === true`	Top-left, violet-500/90 circle, "★" text. Hides on hover (replaced by hover controls).
📌 Pinned icon	`role === "pinned"`	Top-right, indigo-500/90 circle, pin icon. Hides on hover.
▶ Play indicator	`is_video === true`	Center, 40x40px, black/50 circle with white play triangle. Fades out on hover (replaced by live video preview).

Hover overlay:
On hover, a gradient vignette appears (black/80 top, transparent middle, black/80 bottom) and reveals two rows of action buttons:

Top row (appears on hover):

Position	Icon	Button	Action	Backend call
Top-left	✓ checkmark	**Select**	Toggles multi-select checkbox for batch operations (download, delete, publish).	Frontend state only
Top-right #1	📌 Pin icon	**Pin**	Pins this candidate for quick access across shots. Hover: bg turns yellow-500.	`POST /api/shots/{id}/files/{file_id}/pin/`
Top-right #2	⋮ three dots	**More menu**	Opens cascading context dropdown:	

- "Set as Published"
- "Move to Refs"
- ———
- **Quick Actions**
  - ▸ **Crop** → cascading submenu: 1:1, 4:3, 16:9, 9:16, 2.39:1
  - ▸ **Upscale** → cascading submenu: 2x, 4x, Magnific V2
- ———

Quick actions are one-click generation shortcuts: select preset → job starts immediately, new version appears in the shot grid. No modal, no fullscreen required. | Various |

Bottom row (appears on hover):

Position	Icon	Button	Action	Backend call
Bottom-left #1	↻ retry arrow	**Retry**	Copies the exact prompt, model, model settings, prompt tags, references, and elements from this generation directly into the active generation controls pane.	Frontend only — reads all metadata from File record and populates form fields. No backend call required.

Position	Icon	Button	Action	Backend call
			DOES NOT automatically run generation. User can edit settings or change prompt before clicking Generate manually. Only valid for outputs.	
Bottom-left #2	📋 clipboard	**Copy Prompt**	Copies the generation prompt to clipboard. Shows brief "Copied!" toast.	Frontend only — reads `file.prompt`
Bottom-left #3	📎 paperclip	**Use as Reference**	Tags this file as an input ref in the **currently active shot**. No file copying — creates a FileAssignment record. Disabled if already tagged as input in the active shot.	`POST /api/shots/{active_shot_id}/assignments/` with `{ file_id, role: "input" }`
Bottom-right	⬇ download arrow	**Download**	Downloads the original file. Uses presigned Object Storage URL.	`GET /api/files/{file_id}/url/` → browser download

All hover buttons use `stopPropagation` — clicking them does NOT trigger the fullscreen viewer. Only clicking the tile background (not a button) opens fullscreen.

Quick Actions (from three dot menu)

Quick Crop / Quick Upscale are lightweight one-click operations that do NOT open the full Image Lab:

- Clicking a preset immediately dispatches a generation job with default parameters
- The job runs in the background
- A new generation tile appears immediately in the shot grid with loading state
- No modal, no extra steps, no fullscreen required
- Uses the exact same generation pipeline as the Image Lab tools
- Result appears directly in the grid when complete

This is for common operations where the user doesn't need fine control - just a fast one click preset.

Video tile specific behavior:

- On hover: video starts playing (`e.currentTarget.play()`), muted, looped.
- On mouse leave: video pauses and resets to frame 0.
- The center play indicator fades out on hover as the video plays.

In-progress generation tile:

- Shows when a GenerationJob has `status: "processing"` .
- Animated gradient background (dark shimmer).
- Center: "PROCESSING FRAME..." or "RENDERING VIDEO..." text.
- Bottom: progress bar (0-100%) with percentage.
- Disappears and is replaced by the actual output tile when job completes.

Inactive Shot (dimmed)

- Same structure but entire container at `opacity-50` , `hover:opacity-70` .
- Shows both refs and outputs (dimmed). Refs can be useful context even for inactive shots.
- Clicking the header bar or any tile activates the shot (swaps it to full brightness, scrolls to top).

Feature 5: Scene Selector

What it does: Dropdown in the shot grid header bar that lets the user switch between scenes.

```
[<] Scarecrow / [SC03 ▼]  47 shots
                  |
                  ▼
               ┌─────────────────┐
               | SC01 Opening    |
               | SC02 Chase      |
               | SC03 ✓ Conf...  |
               | SC04 Escape     |
               | SC05 Finale     |
               └─────────────────┘
```

- Native `<select>` element bound to `activeSceneId` state.
- On change: fetches the new scene's shots from `GET /api/scenes/{scene_id}/shots/` .
- Auto-activates the first shot in the new scene.
- Shot count updates to reflect the new scene.

Feature 6: Fullscreen Viewer — Image Lab / Video Lab

What it does: Clicking any candidate tile opens a fullscreen overlay portal. The overlay detects file type and opens the appropriate lab:

- **Image files** → Image Lab (non-destructive layered editing with 7 tool modes and version history)
- **Video files** → Video Lab (playback suite with frame extraction, re-generation tools, and version history)

Both labs share the same version history system, navigation controls, and the "save to workspace" workflow.

Reference: `src/components/ImageEditor.tsx` , `src/components/VideoPlayer.tsx`

CRITICAL: Rendered via `createPortal(component, document.body)` to escape parent overflow/z-index stacking. Without the portal, the viewer gets clipped by the grid's scroll container.

6.1 Shared Layout — The Lab Shell

Every lab instance renders inside this shell:

```
┌──────────────────────────────────────────────────────────────────────────┐
│ [◀ prev]        IMAGE LAB  ·  Shot 03 – Frame 7         [next ▶]  [X]   │
├──────┬──────────────────────────────────────────────┬──────────────────┤
│      │                                              │ VERSION HISTORY  │
│ TOOL │                                              │                  │
│ BAR  │              MAIN WORKSPACE                  │ ┌──────────────┐ │
│      │              (zoomable, pannable)            │ │ V1 Original  │ │
│ ┌──┐ │                                              │ │ V2 Crop   ★  │ │
│ │✏️│ │         ┌──────────────────┐                 │ │ V3 Inpaint   │ │
│ │⬜│ │         │                  │                 │ │ V4 Upscale   │ │
│ │🖌️│ │         │   active version  │                 │ │              │ │
│ │🖼️│ │         │     displayed    │                 │ │              │ │
│ │✨│ │         │                  │                 │ └──────────────┘ │
│ │⬆│ │         └──────────────────┘                 │                  │
│ │🧩│ │                                              │                  │
│ └──┘ │                                              │                  │
├──────┴──────────────────────────────────────────────┴──────────────────┤
│                        BOTTOM TOOLBAR                                   │
│  (contextual – changes based on selected tool mode)                     │
└──────────────────────────────────────────────────────────────────────────┘
```

Shell controls (always visible):

Control	Key/Click	Action
Close	X button / Escape	Exit lab. Unsaved versions are discarded (confirmation dialog if unsaved edits exist).
Previous	◀ button / Left arrow	Navigate to previous candidate in the shot (loads that file's version tree).
Next	▶ button / Right arrow	Navigate to next candidate.
Zoom	Scroll wheel / pinch	Zoom in/out on the workspace.
Pan	Click + drag (when zoomed)	Pan the canvas.
Fit	Double-click canvas	Reset zoom to fit-to-viewport.

6.2 Version History System

The version history is the core concept of both labs. Every edit creates a new **immutable version** — the original is never modified.

Version History Panel (right side, always visible)

```
┌──────────────────────────┐
│ VERSION HISTORY        ↻ │
│                          │
│ ┌────────────────────┐   │
│ │ ● V1  Original   💾 │ │  ← green left border = saved to workspace
│ └────────────────────┘   │
│       │                  │
│       ├── V2  Crop       │
│       │     │            │
│       │     └── V3  Inpaint │  ← branched from V2
│       │                  │
│       └── V4  Vary     💾 │  ← also saved (branched from V1)
│                          │
│ ┌────────────────────┐   │
│ │ ● V5  ◌ Processing...  │ │  ← pulsing orange dot, loading state
│ └────────────────────┘   │
└──────────────────────────┘
```

Version entry — each row shows:

Element	Description
Version number	Sequential (V1, V2, V3...)
Mode badge	Which tool created it (Crop, Inpaint, Vary, etc.)
Tree indent	Visual parent-child relationship. Lines connect parent → child.
Active highlight	Orange border + subtle orange background on the currently viewed version.
Saved indicator	Green left border (#4ADE80) + 💾 icon on versions saved to workspace.
Processing state	Pulsing orange dot + "Processing..." text for in-progress generations.
Hover actions	`[💾 Save]` `[📋 Copy]` `[⬇ Download]` appear on hover.

Version entry hover actions:

Button	Action
💾 **Save**	Promotes this version to the workspace. Creates a `FileAssignment` with `role="output"` for the current shot. Green accent applied.
📋 **Copy**	Copies the version image/video to clipboard.
⬇ **Download**	Downloads the version file to local disk.

Version Data Model — How Versions Map to Files

Each version is a `File` record in the database. The version tree is built from the `parent_file` foreign key.

```
File record for a version:
{
  id:              "file_uuid_v3",
  project:         <project_id>,
  s3_key:          "generations/<project_id>/file_uuid_v3.png",
  origin:          "edit",                        ← distinguishes lab edits from original generatior
  parent_file:     "file_uuid_v2",                ← parent version (V2 in this case)
  model_id:        "flux-inpaint-v1",             ← which AI model processed this edit
  prompt:          "remove the car from the road", ← edit prompt (if applicable)
  model_settings:  { "brush_size": 40, "mask_mode": "paint" },  ← tool-specific params frozen at edi
  width:           1920,
  height:          1080,
  extension:       "png",
  is_video:        false,
  created_at:      "2026-03-31T14:20:00Z",
  deleted_at:      null
}
```

Tree reconstruction query:

```sql
-- Get all versions for a source file (recursive)
WITH RECURSIVE version_tree AS (
  SELECT * FROM files WHERE id = '<original_file_id>'
  UNION ALL
  SELECT f.* FROM files f
  JOIN version_tree vt ON f.parent_file_id = vt.id
  WHERE f.deleted_at IS NULL AND f.origin = 'edit'
)
SELECT * FROM version_tree ORDER BY created_at;
```

Processing Cycle

When the user triggers a generative edit (Inpaint, Prompt Edit, Vary, Upscale, Composer):

```
1. USER clicks "Generate" in the bottom toolbar
       │
2. FRONTEND creates a temporary version entry in the version panel
   ├── Pulsing orange dot
   ├── "Processing..." label
   └── Version number assigned immediately (optimistic)
       │
3. FRONTEND sends POST /api/generate with:
   {
     modelId:        "flux-inpaint-v1",
     inputSnapshot: {
       params: {
         prompt:     "remove the car",
         brush_size: 40,
         mask_data:  "<base64 mask>"
       },
       assetRefs: [
         { portKey: "source_image", assetId: "<parent_version_file_id>" }
       ]
     },
     context: {
       lab_mode:       "inpaint",
       parent_file_id: "<parent_version_file_id>",
       shot_id:        "<current_shot_id>"
     }
   }
       │
4. BACKEND executes the universal generate pipeline:
   validate → createJob → buildPayload → resolveAssets → bridge.submit → bridge.poll → saveAsset → u
       │
5. BACKEND on completion:
   ├── Creates File record with origin="edit", parent_file=<parent_version_file_id>
   ├── Creates FileAssignment { file, shot, role="output" } ONLY if user explicitly saves
   └── Returns { jobId, fileId, s3_url }
       │
6. FRONTEND (polling GET /api/jobs/:jobId):
   ├── Detects status="completed"
   ├── Replaces the processing placeholder with the real version entry
   ├── Auto-selects the new version in the workspace
   └── Displays the result image/video in the main canvas
```

Branching

Branching happens automatically when the user selects a non-latest version and edits it:

```
V1 (Original)
├── V2 (Crop from V1)
│   └── V3 (Inpaint from V2)
└── V4 (Vary from V1)        ← user clicked V1, then used Vary → branch
    └── V5 (Upscale from V4) ← linear continuation of the V4 branch
```

The `parent_file` FK captures this automatically. The frontend renders the tree by walking `parent_file` relationships.

Save-to-Workspace Flow (The "Green" State)

- **Default:** Lab versions exist only as `File` records with `origin="edit"`. They have NO `FileAssignment` to any shot — they are "lab-only" files.
- **Save action:** Clicking 💾 on a version creates a `FileAssignment { file: <version_file>, shot: <current_shot>, role: "output" }`. This makes the version appear in the shot's output grid.
- **Visual feedback:** Saved versions get a green left border (#4ADE80) and the 💾 icon fills in.
- **Multiple saves:** The user can save multiple versions from the same tree. Each becomes a separate output in the shot.
- **Exit behavior:** On closing the lab, unsaved versions remain as `File` records (not deleted) but are not visible anywhere in the main UI since they have no `FileAssignment`. They can be found via the file's version tree if the user re-opens the parent file in the lab.

6.3 Image Lab — 7 Tool Modes

The left toolbar shows 7 tool icons. Clicking one activates that mode, which changes:

1. The cursor behavior on the canvas
2. The bottom toolbar contents
3. What happens when "Generate" is clicked

Mode 1: Annotate (✏️ Pencil icon)

What it does: Non-destructive freehand drawing, shapes, and text overlaid on the image. Does NOT call an AI model — this is a local-only tool for marking up images (e.g., director notes, VFX callouts).

Canvas behavior: Direct drawing on the canvas with the selected tool.

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────┐
│  [◎ Point] [━ Line] [○ Shape] [Ty Text]  │  Color: [●]  Size: [━━●━━]  │  [Cancel] [Save ✓] │
└──────────────────────────────────────────────────────────────────────┘
```

Control	Description
◎ **Point**	Click to place a numbered marker point on the canvas.
— Line	Click and drag to draw a freehand line. Stroke color and width from Color/Size controls.
○ **Shape**	Click and drag to draw rectangles, circles, or arrows. Submenu for shape type.
Ty Text	Click to place a text box. Type to enter text. Font: Inter, size adjustable.
Color picker	Circular swatch — opens a color palette (preset colors + custom hex). Default: #F7931A (orange).
Size slider	Brush/stroke width. Range: 1px – 20px. Default: 3px.
Cancel	Discards all annotations made in this session (before Save).
Save ✓	Flattens annotations onto the image, creates a new version with `mode: "annotate"`. Local operation — no API call.

Mode 2: Crop (Crop icon)

What it does: Non-destructive crop with preset and freeform aspect ratios. Local operation — no AI model.

Canvas behavior: Draggable crop handles appear over the image. The area outside the crop region is dimmed (semi-transparent black overlay).

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────┐
│  Aspect: [Free ▼]  │  Presets: [1:1] [4:3] [16:9] [9:16] [2.39:1]    │  [Cancel] [Apply ✓] │
└──────────────────────────────────────────────────────────────────────┘
```

Control	Description
Aspect dropdown	Free, 1:1, 4:3, 3:4, 16:9, 9:16, 2.39:1 (anamorphic), 1.85:1 (theatrical).
Preset pills	Quick-select common ratios. Active preset gets orange border.
Crop handles	8 handles (4 corners + 4 edges). Drag to resize. Hold Shift to constrain.
Rule-of-thirds overlay	Faint grid overlay inside crop area for composition guidance.
Cancel	Removes crop overlay, returns to previous state.
Apply ✓	Creates a new version with the cropped dimensions. Updates `width` / `height` on the File record. Local operation.

Mode 3: Inpaint (🖌️ Brush icon)

What it does: Paint a mask region over the image, then provide a prompt describing what should fill that region. AI generates the fill. This is a **generative mode** — it calls the universal generate pipeline.

Canvas behavior: Semi-transparent red/orange overlay appears where the user paints. The mask is a separate layer.

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  Model: [Flux Fill ▼]  │  [🖌️ Paint] [□ Rect] [⊘ Erase]  │  Size: [━━●━━]  │         │
│                        │                                  │                 │         │
│  Prompt: [> describe what to fill in the masked area____________________________] [GENERATE] │
└──────────────────────────────────────────────────────────────────────────────┘
```

Control	Description
Model selector	Dropdown of inpainting-capable models. Filtered by `capabilities: ["inpaint"]`. Default: Flux Fill. Shows model name + credit cost.
🖌️ **Paint**	Freehand brush mode. Paint mask directly on canvas.
□ **Rect**	Rectangle mask mode. Click and drag to draw a rectangular mask region.
⊘ **Erase**	Erase parts of the mask. Same brush size control.
Size slider	Mask brush diameter. Range: 5px – 200px. Default: 40px.
Prompt input	Text input for describing the fill content. Orange border on focus. Supports `@Element` autocomplete (same as main prompt).
GENERATE	Orange gradient pill button. Sends the source image + mask + prompt to the inpaint model via the generate pipeline. Creates a new version on completion.

Generate payload (sent to backend):

```json
{
  "modelId": "flux-fill-v1",
  "inputSnapshot": {
    "params": {
      "prompt": "a red sports car parked on the street",
      "mask_data": "<base64-encoded mask PNG — white=fill, black=keep>"
    },
    "assetRefs": [
      { "portKey": "source_image", "assetId": "<current_version_file_id>" }
    ]
  }
}
```

Mode 4: Prompt Edit (🖼️ Frame icon)

What it does: Select a rectangular region, then write a prompt to modify the content within that region. Unlike inpaint, this mode **replaces** the selected area with generated content guided by the prompt — it doesn't just fill in masked gaps.

Canvas behavior: Click and drag to draw a selection rectangle. The selected area is highlighted with an orange dashed border.

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────────┐
│  Model: [Flux Edit ▼]  │  [↔ Resize: ON]  │  Strength: [━━━●━━]                    │
│                                                                                      │
│  Prompt: [> describe what this region should become_____________________________] [GENERATE] │
└──────────────────────────────────────────────────────────────────────────┘
```

Control	Description
Model selector	Dropdown filtered by `capabilities: ["img2img", "region_edit"]` . Default: Flux Edit.
↔ Resize toggle	When ON, the output can have different dimensions than the selection region (the model decides). When OFF, output matches selection exactly.
Strength slider	How much to deviate from the original content. Range: 0.0 (preserve original) – 1.0 (full replacement). Default: 0.7.
Prompt input	Describe the desired replacement. Orange border on focus. `@Element` autocomplete supported.
GENERATE	Sends region coordinates + source image + prompt to the edit model. Creates a new version.

Mode 5: Vary (✨ Sparkle icon)

What it does: Generate variations of the entire image. The model takes the current version as input and produces a similar-but-different output. Useful for exploring alternatives without re-prompting from scratch.

Canvas behavior: No special cursor — the entire image is the input.

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────────┐
│  Model: [Flux Redux ▼]  │  Strength: [━━━●━━]  │  [🔍 Describe]                    │
│                                                                                      │
│  Prompt: [> optional: guide the variation direction_____________________________] [GENERATE] │
└──────────────────────────────────────────────────────────────────────────┘
```

Control	Description
Model selector	Dropdown filtered by `capabilities: ["variation", "img2img"]`. Default: Flux Redux.
Strength slider	How different the variation should be. Range: 0.1 – 1.0. Low = subtle changes, High = dramatic reinterpretation. Default: 0.5.
🔍 **Describe**	Reverse-prompts the current image using a VLM (vision-language model). Populates the prompt field with a detailed description. Useful as a starting point before tweaking. Calls `POST /api/describe` with the image.
Prompt input	Optional — if provided, guides the variation direction. If empty, the model uses only the image as input.
GENERATE	Sends source image + optional prompt + strength to the variation model. Creates a new version.

Mode 6: Upscale (⬆ Arrow icon)

What it does: Increase the resolution of the current version. AI-powered super-resolution that adds detail.

Canvas behavior: No special cursor. A comparison overlay shows original vs upscaled on completion (split-view slider).

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────────┐
│  Model: [Aura SR v2 ▼]  │  Target: [4K UHD ▼]  │  Creativity: [──●──]              │
│                                                                          │
│  Current: 1024×768  →  Target: 3840×2160  │  Scale: 3.75x                  [GENERATE]  │
└──────────────────────────────────────────────────────────────────────────┘
```

Control	Description
Model selector	Dropdown filtered by `capabilities: ["upscale"]`. Options: Aura SR v2, Magnific v2, Real-ESRGAN. Shows credit cost per model.
Target dropdown	Preset output resolutions: 2K (2048×1536), 4K UHD (3840×2160), 8K (7680×4320), Custom. "Custom" reveals width/height number inputs.
Creativity slider	How much the model "halluccinates" new detail vs just interpolating. Range: 0.0 – 1.0. Default: 0.3. Only shown for models that support it (e.g., Magnific). Hidden via `showWhen` for models that don't.
Resolution preview	Shows current dimensions → target dimensions and the scale factor. Updates dynamically when target changes.

Control	Description
GENERATE	Sends source image + target resolution + creativity to the upscale model. Creates a new version with updated `width` / `height` .

Mode 7: Composer (🧩 Puzzle icon)

What it does: Free canvas collage and reference layering mode. Drag any image from any shot onto the canvas, position, scale, rotate, and arrange them for AI composition. This is for creating multi-reference collages, shot mashups, and precisely arranging reference positions. **NOT just face/object replacement.**

Canvas behavior:

- Canvas is the full image area
- Drag any asset thumbnail from the asset browser onto the canvas
- Once placed, every asset is a draggable, resizable, rotatable layer
- Each layer has corner handles for resize, rotation handle at the top
- Layers can be reordered (bring forward / send backward) via right click
- Unlimited number of layers can be placed
- All layers are sent as IP-Adapter references when generating

Bottom toolbar — split into two rows:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  Model: [Flux Composer ▼]  │  [Clear Canvas]  [Reset]  [Generate]            │
├──────────────────────────────────────────────────────────────────────────────┤
│  ⬅ ──────────────────────────────────────────────────────────────────── ➡   │
│  🟡 Current Shot   ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐   │
│                    │ img  │ │ img  │ │ img  │ │ img  │ │ img  │ │ img  │ ...│
│                    └──────┘ └──────┘ └──────┘ └──────┘ └──────┘ └──────┘   │
│                                                                              │
│  SH01   ┌──────┐ ┌──────┐ ┌──────┐                                          │
│         │ img  │ │ img  │ │ img  │  ...                                     │
│         └──────┘ └──────┘ └──────┘                                          │
│                                                                              │
│  SH02   ┌──────┐ ┌──────┐                                                   │
│         │ img  │ │ img  │  ...                                              │
│         └──────┘ └──────┘                                                   │
│                                                                              │
│  [View All Assets] ──────────────────────────────────────────────────── ▼   │
└──────────────────────────────────────────────────────────────────────────────┘
```

Asset Browser (bottom row):

- **Horizontally scrollable** container that loads assets lazily
- First section always shows assets from **the current shot** (highlighted yellow)
- Then ordered by shot sequence: SH01, SH02, SH03, etc.

- Click `[View All Assets]` to open the full screen asset browser modal that shows every asset in the entire project

Control	Description
Drag & Drop	Drag any asset thumbnail onto the canvas to add it as a new layer
Layer controls	Click a layer on the canvas to select it. Drag to move. Drag corner handles to resize. Drag top handle to rotate.
Right click menu	Bring forward, Send backward, Delete layer, Reset transform
Clear Canvas	Removes all layers, resets to empty
Reset	Resets all layers back to original position/size
Generate	Sends base image + all placed layers + their positions, sizes, opacity values to the composer model. Creates a new version.

This is how you make collages of shots, combine reference images, position elements exactly where you want them, and build complex multi-reference compositions before generation. All layers are passed as reference images to the AI with their exact canvas positions.

Composer generate payload:

```
{
  "modelId": "flux-composer-v1",
  "inputSnapshot": {
    "params": {
      "bbox": { "x": 120, "y": 80, "width": 200, "height": 280 },
      "strength": 0.8,
      "style_mix": 0.3
    },
    "assetRefs": [
      { "portKey": "source_image", "assetId": "<current_version_file_id>" },
      { "portKey": "element_ref", "assetId": "<element_frontal_image_file_id>" }
    ]
  }
}
```

6.4 Video Lab

The Video Lab opens when clicking a video candidate tile. It shares the same shell layout and version history system as the Image Lab, but the left toolbar and bottom toolbar are different.

Video Lab Layout

```
┌──────────────────────────────────────────────────────────────────────────┐
│ [◀ prev]        VIDEO LAB  ·  Shot 03 – Clip 2          [next ▶]  [X]    │
├──────┬──────────────────────────────────────────────┬──────────────────┤
│      │                                              │ VERSION HISTORY  │
│ TOOL │                                              │                  │
│ BAR  │              VIDEO PLAYBACK                  │ ┌──────────────┐ │
│      │              (zoomable, pannable)            │ │ V1 Original  │ │
│ ┌──┐ │                                              │ │ V2 Trim   ★  │ │
│ │▶️│ │         ┌──────────────────┐                 │ │ V3 Extend    │ │
│ │✂️│ │         │                  │                 │ │              │ │
│ │🔄│ │         │   active version  │                 │ │              │ │
│ │📷│ │         │     playing      │                 │ └──────────────┘ │
│ │⬆️│ │         │                  │                │                  │
│ │🎵│ │         └──────────────────┘                 │                  │
│ └──┘ │                                              │                  │
│      │  ──●──────────────────── 00:02 / 00:05       │                  │
├──────┴──────────────────────────────────────────────┴──────────────────┤
│                        BOTTOM TOOLBAR                                   │
│  (contextual – changes based on selected tool mode)                     │
└──────────────────────────────────────────────────────────────────────────┘
```

Video Lab Tool Modes

Left toolbar (5 tools):

Icon	Tool	Type	Description
▶️ Play	**Review**	Local	Default mode. Full playback controls: play/pause, frame step, speed control, timeline scrubber.
✂️ Scissors	**Trim**	Local	Set in/out points on the timeline to trim the video. Non-destructive — creates a new version with updated duration.
📷 Extract	**Frame Extract**	Local	Click to extract the current frame as a PNG image. Creates a new `File` with `origin="extraction"`, `is_video=false`. Optionally assigns it as an output to the current shot.
⬆️ Upscale	**Video Upscale**	Generative	AI-powered video super-resolution. Same concept as Image Lab upscale but for video.
🎵 Audio	**Audio Detach**	Local	Strips the audio track from the video into a separate audio `File`. The video version continues without audio.

Mode: Review (▶️) — Default

Bottom toolbar (always visible in Review mode):

```
┌──────────────────────────────────────────────────────────────────────┐
│  ──●──────────────────────────────────── 00:02 / 00:05                │
│  [⏮] [▶] [⏭]  │  [1x ▼]  │  [📷 Extract Frame]  │  [⬇ Download]     │
└──────────────────────────────────────────────────────────────────────┘
```

Control	Key/Click	Action
Play/Pause	▶ button / Spacebar	Toggle playback.
Frame step back	⏮ / , (comma)	Step back one frame (1/fps seconds).
Frame step forward	⏭ / . (period)	Step forward one frame.
Speed dropdown	[1x ▼]	0.25x, 0.5x, 1x, 1.5x, 2x.
Timeline scrubber	Click/drag	Seek to position. Shows frame number on hover.
Extract Frame	📷 button	Captures current frame as PNG. Quick shortcut — same as the Extract tool mode but one-click.
Download	⬇ button	Downloads the video file to local disk.

Mode: Trim (✂️)

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────┐
│  [IN ●]──────────────────────────[OUT ●]──────────────                │
│  In: 00:00.500   Out: 00:03.200   Duration: 2.7s                      │
│  [▶ Preview]  │  [Set In] [Set Out]  │  [Cancel] [Apply Trim ✓]       │
└──────────────────────────────────────────────────────────────────────┘
```

Control	Description
In/Out markers	Draggable markers on the timeline. Orange handles.
Set In	Sets the in-point to the current playhead position. Keyboard: `I`.
Set Out	Sets the out-point to the current playhead position. Keyboard: `O`.
Preview	Plays only the trimmed segment in a loop.
Duration display	Shows the resulting clip duration. Updates as markers move.

Control	Description
Apply Trim	Creates a new version with the trimmed segment. Local operation using ffmpeg-wasm (client-side). Updates `duration` on the File record.

Mode: Frame Extract (📷)

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────┐
│  Frame: 47 / 150  │  Time: 00:01.567  │  Format: [PNG ▼]              │
│  [◀ -1] [▶ +1]  │  [💾 Save to Shot] [⬇ Download] [📋 Copy]          │
└──────────────────────────────────────────────────────────────────────┘
```

Control	Description
Frame counter	Shows current frame number / total frames.
Frame step	◀ -1 and ▶ +1 buttons (or arrow keys) to step frame by frame.
Format dropdown	PNG (lossless) or JPEG (smaller). Default: PNG.
Save to Shot	Extracts the frame, creates a `File` with `origin="extraction"` and `is_video=false`, and creates a `FileAssignment { shot, role="output" }`. The extracted frame appears in the shot's grid.
Download	Extracts and downloads without saving to the project.
Copy	Extracts and copies to clipboard.

Mode: Video Upscale (⬆)

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  Model: [Topaz Video AI ▼]  │  Target: [4K ▼]  │  Denoise: [──●──]  │  FPS: [Keep ▼]  │
│                                                                              │
│  Current: 1280×720 @ 30fps  →  Target: 3840×2160 @ 30fps  │  Est. time: ~3 min  [GENERATE] │
└──────────────────────────────────────────────────────────────────────────────┘
```

Control	Description
Model selector	Dropdown filtered by `capabilities: ["video_upscale"]`.
Target dropdown	2K, 4K, 8K preset resolutions.
Denoise slider	Noise reduction strength. Range: 0.0 – 1.0. Default: 0.3.

Control	Description
FPS dropdown	Keep (same as source), 30fps, 60fps (frame interpolation).
Estimate	Shows estimated processing time based on video duration × resolution.
GENERATE	Sends video to upscale model via generate pipeline. Creates a new version.

Mode: Audio Detach (🎵)

Bottom toolbar:

```
┌──────────────────────────────────────────────────────────────────────────┐
│  Audio track detected: AAC 48kHz stereo  │  Duration: 5.0s               │
│  [🎵 Extract Audio] [🔇 Remove Audio]  │  [⬇ Download Audio]           │
└──────────────────────────────────────────────────────────────────────────┘
```

Control	Description
Track info	Shows detected audio codec, sample rate, channels. "No audio track" if silent.
Extract Audio	Creates a separate audio `File` with `origin="extraction"` , `is_audio=true` . Saves to project.
Remove Audio	Creates a new video version with the audio track stripped.
Download Audio	Downloads just the audio track as WAV/MP3.

6.5 Generation Pipeline Integration

Both labs use the same universal generate pipeline described in `docs/backend-refactor.md` and `docs/generate-flow.html` . The pipeline is:

```
validate → createJob → buildPayload → resolveAssets → bridge.submit → bridge.poll → saveAsset → upda
```

Lab-specific additions to the pipeline:

Step	Standard Generation	Lab Generation
Input	User configures from controls pane	Lab provides source image/video + tool-specific params
Job creation	`inputSnapshot` captures prompt + refs + settings	Same, plus `parent_file_id` in context

Step	Standard Generation	Lab Generation
Asset save	Creates `File` with `origin="generation"`	Creates `File` with `origin="edit"` and `parent_file=<parent_version>`
FileAssignment	Automatically creates `{ shot, role="output" }`	NO automatic assignment — only on explicit save-to-workspace
UI update	New tile appears in shot grid	New version appears in version history panel

Lab models are the same model registry — they're just filtered by capabilities:

- Inpaint: `capabilities: ["inpaint"]`
- Prompt Edit: `capabilities: ["img2img", "region_edit"]`
- Vary: `capabilities: ["variation", "img2img"]`
- Upscale (image): `capabilities: ["upscale"]`
- Upscale (video): `capabilities: ["video_upscale"]`
- Composer: `capabilities: ["compose", "face_swap", "ip_adapter"]`

`deriveJob()` for Re-Generate mode:

```
deriveJob(originalJobId, overrides) →
  1. Reads original job's inputSnapshot
  2. Merges overrides (changed prompt, seed, model, etc.)
  3. Calls generate() with new inputSnapshot + parentJobId link
  4. New File gets parent_file = original video's file ID
```

Feature 7: Generation Controls — Image Tab

What it does: The controls pane when the Image mode tab is active. Artist selects a model, adds references, writes a prompt, configures cinematography settings, sets parameters, and generates.

Reference: `src/components/ControlsPane.tsx` lines 2965-3580

Layout (top to bottom):

```
┌──────────────────────────────────────┐
│ [Flux 1.1 Pro Ultra            ▼]    │  ← Model selector
├──────────────────────────────────────┤
│ REFERENCE IMAGES (OPTIONAL)  Max 6   │
│ ┌────┐ ┌────┐ ┌────┐ [+]            │  ← Reference uploads
│ │ref1│ │ref2│ │ref3│                 │
│ └────┘ └────┘ └────┘                 │
├──────────────────────────────────────┤
│ ELEMENTS (FACE CONTROL)              │  ← Only for supporting models
│ ┌────┐ ┌────┐                        │
│ │E1  │ │E2  │   [+ Add from Elements]│
│ └────┘ └────┘                        │
├──────────────────────────────────────┤
│ ┌──────────────────────────────┐     │
│ │                          [👁] │     │  ← Reveal Tags icon (top-right)
│ │ > A weathered scarecrow with  │     │  ← Prompt textarea
│ │   tattered burlap clothing...  │     │
│ │                               │     │
│ │ [↻][↷][🗑]              [✨ ▾] │     │  ← Bottom toolbar
│ └──────────────────────────────┘     │
│                                      │
│ 🎬 CINEMATOGRAPHER          [ON 🔘] │  ← Toggle switch
│ [ Alexa 65 · S7/i · 35mm · f/1.4 · Golden ] │  ← Orange pills
├──────────────────────────────────────┤
│ ┌──────────────┐  ┌────────────────┐  │
│ │ ASPECT       │  │ RESOLUTION     │  │
│ │ 16:9 Cinema  │  │ 1080p          │  │
│ └──────────────┘  └────────────────┘  │
│ ┌──────────────┐  ┌────────────────┐  │  ← Model-specific
│ │ CFG SCALE    │  │ SEED           │  │    (shown per model)
│ │ 7.5          │  │ Auto           │  │
│ └──────────────┘  └────────────────┘  │
│ Advanced Settings                  ▸  │  ← Collapsible
├──────────────────────────────────────┤
│                                      │
│ [▇▇] [▇▇▇▇ GENERATE ▇▇▇▇]  [1 ▾]     │  ← Sticky footer
└──────────────────────────────────────┘
```

7a. Model Selector

- Full-width dropdown (`<select>`) with `rounded-[20px]` .
- Groups models under "Image Pipelines" optgroup.
- Each option shows: model name (e.g. "Flux 1.1 Pro Ultra").
- Disabled during generation (opacity-50).
- **Tab memory:** Switching away and back remembers the last selected model per tab.

Available image models (configurable):

Model ID	Label	Provider	Supports Elements	Supports References	Max Refs
`flux-1.1-pro-ultra`	Flux 1.1 Pro Ultra	FAL	No	Yes (optional)	4
`flux-1.1-pro`	Flux 1.1 Pro	FAL	No	Yes (optional)	4
`kling-3.0-image`	Kling 3.0 Image	KIE	Yes	Yes	4
`ideogram-v2`	Ideogram V2	FAL	No	No	0
`sdxl-turbo`	SDXL Turbo	FAL	No	Yes (required)	1

7b. Reference Images

- **Header:** "REFERENCES" with "+ Add" link top-right.
- **Max count** shown top-right: "Max 4" (varies by model).
- **Drop zone:** Rounded panel. Accepts drag & drop of images, element character sheets, or file entries from the shot grid.
- **Thumbnails:** 40x40px rounded squares. Hover reveals X button to remove. Shows upload spinner while uploading.
- **[+] button:** Opens file picker for image files. Only shown if under the limit.
- **Empty state:** "Drag images or element sheets, or click +"

7c. Elements (Character References)

- **Always shown in Image mode** — not model-specific. Elements work across all image models by injecting the element's frontal character sheet as a reference image. This is not the Kling-specific element API; it simply uses the character sheet image as an additional reference.
- **Label:** "Elements" with "+ Add from Elements" link.
- Opens the Elements Manager panel (a slide-out that lists all project elements with their frontal images).
- Selected elements appear as 56x56px thumbnails with:
  - "E1", "E2" badge top-left (amber background).
  - Element name bottom.
  - X button on hover to deselect.
- **Usage:** Reference elements in the prompt with `@Element1`, `@Element2`, etc. (via @ autocomplete).

7d. Prompt Textarea

- 6-row textarea with `rounded-[24px]`.
- **@ autocomplete:** Typing `@` triggers a popup listing all available references and elements. The popup shows:
  - `@img1`, `@img2`, etc. — for each uploaded reference image (thumbnail + filename).
  - `@Element1`, `@Element2`, etc. — for each pinned/selected element (frontal thumbnail + element name).
  - Navigate with arrow keys, select with Enter/Tab. Grouped under "References" and "Elements" headers in the popup.

- 👁️ **Reveal Tags (top-right corner):** Eye icon button positioned in the top-right of the textarea. When clicked:
  i. Appends the current cinematographer prompt tags (e.g., `[ Cinematic, ARRI Alexa 65, 35mm, f/1.4 ]`) directly into the prompt textarea text, after the main prompt.
  ii. Automatically disables the Cinematographer ON/OFF toggle to prevent the tags from being duplicated (appended twice) when generating.
  iii. The inlined tags are rendered as styled orange text within the textarea to visually distinguish them from the user's main prompt.
  iv. Clicking the eye icon again removes the inlined tags from the text and re-enables the Cinematographer toggle.

Bottom toolbar inside textarea:

Position	Control	Description
Bottom-left #1	↻ **Undo**	Reverts to previous prompt. Subtle icon button. Disabled when no history.
Bottom-left #2	↱ **Redo**	Re-applies undone prompt. Subtle icon button. Disabled when nothing to redo.
Bottom-left #3	🗑️ **Clear**	Clears the entire prompt textarea. Subtle icon button. Disabled when prompt is empty.
Bottom-right	✨ **Auto-Expand (▾ dropdown)**	Sparkle icon with a dropdown chevron. Click the icon to send the prompt to an LLM to expand into a detailed cinematic description. Click the chevron to open a dropdown of expansion templates (e.g., "Cinematic", "Editorial", "Anime", "Photoreal"). Selecting a template applies that expansion style. Shows spinner while processing. Disabled if prompt is empty.

7e. Cinematographer Panel

Trigger: The `ImageShotLookCards` component renders below the prompt as a distinct row. Shows a 🎬 camera icon, "CINEMATOGRAPHER" label, and an **ON / OFF toggle switch** on the right side. When ON, cinematographer settings are appended to the prompt automatically and the toggle glows orange. Below the row, current selections are displayed as orange pill badges (e.g., `Alexa 65 · S7/i · 35mm · f/1.4 · Golden`).

Clicking the row opens the Cinematographer modal:

```
┌──────────────────────────────────────────┐
│ CINEMATOGRAPHER (gradient text)          × │
├──────────┬────────────┬─────────┬──────────┤
│  CAMERA  │  LIGHTING  │  STYLE  │ HEXCODES │ ← 4 pill tabs
├──────────┴────────────┴─────────┴──────────┤
│                                          │
│ Camera tab: 4 scrollable columns         │
│ ┌──────────┬──────────┬─────────┬────────┐│
│ │ Body     │ Lens     │ Focal   │ f-Stop │
│ │          │          │         │        │
│ │*ARRI 65*│*Cooke*   │ 14mm    │ f/1.2  │
│ │ RED 8K   │*S7/i*    │ 24mm    │*f/1.4* │
│ │ Sony V2  │ Zeiss    │*35mm*   │ f/2    │
│ │ Panav    │ Primo    │ 50mm    │ f/2.8  │
│ │ BMPCC    │ ARRI Sig│ 85mm    │ f/4    │
│ │ Canon    │ Leica M  │ 100mm   │ f/5.6  │
│ │          │          │ 135mm   │ f/8    │
│ │          │          │ 200mm   │ f/11   │
│ └──────────┴──────────┴─────────┴────────┘│
│                                          │
├──────────────────────────────────────────┤
│ ARRI Alexa 65 · Cooke S7/i · 35mm ·     │
│ f/1.4                                    │
│                    [Reset]  [Apply ▸]    │
└──────────────────────────────────────────┘
```

Camera tab — 4 columns, cascading selection (340px height, scrollable):

Selection is **cascading**: the artist first picks a camera body. Once a body is selected, the Lens column populates with compatible lens options for that camera system. Focal Length and f-Stop columns are always visible and independent — they do not depend on the Body or Lens selection.

Column	Behavior	Options
Body	Always visible. Select first.	ARRI Alexa 65, RED Raptor 8K, Sony Venice 2, Panavision DXL2, Blackmagic URSA 12K, Canon C500 MkII, IMAX MSMD
Lens	**Appears after Body is selected.** Shows lenses compatible with the chosen camera system.	Cooke S7/i, Zeiss SP, Primo 70, ARRI Signature, Atlas 1.5x Anamorphic, Leica M (filtered by body)
Focal Length	Always visible. Independent.	14mm, 24mm, 35mm, 50mm, 85mm, 100mm, 135mm, 200mm
f-Stop	Always visible. Independent.	f/1.2, f/1.4, f/2, f/2.8, f/4, f/5.6, f/8, f/11

Active items: orange border + orange glow shadow.

Lighting tab — 3x3 grid of large square visual panels, each with a descriptive icon and label:

Each panel is a ~140x140px rounded square card (`rounded-[16px]`). Contains an **actual photograph/image** that visually demonstrates the lighting style, with a label overlaid at the bottom. These are not icons — they are real reference images showing the lighting effect applied to a scene. Active panel: golden border + golden-tinted overlay.

Panel	Image	Description
Golden Hour	Photo of a scene bathed in warm golden sunset light	Warm sunset tones
Blue Hour	Photo of a scene in cool blue pre-dawn light	Cool pre-dawn tones
Rembrandt	Photo with classic Rembrandt triangle lighting	Classic portrait lighting
Overcast	Photo of a scene in soft, even overcast light	Soft diffused daylight
Low Key	Photo of a dark, moody scene with dramatic shadows	Dark, dramatic shadows
Practical	Photo of a scene lit by visible in-frame light sources	Visible in-scene light sources
Moonlight	Photo of a scene in cold silver moonlight	Cold silver light
Neon	Photo of a scene with vibrant neon-colored lights	Cyberpunk colored lights
High Key	Photo of a bright, evenly lit scene with minimal shadows	Bright, minimal shadows

Style tab — Grid of large square visual panels with actual images, organized in 2 sections (Films + Fashion):

Same card style as Lighting — each panel shows an **actual photograph** demonstrating the film or fashion look, with a label overlaid at the bottom. Active panel: cyan/blue border + subtle tinted overlay.

Films section (1x4 row):

Panel	Image	Description
Amélie	Photo in warm, whimsical Amélie color grading	Whimsical warm French tones
Mad Max	Photo in desaturated, orange-heavy desert tones	Desaturated orange desert tones
Matrix	Photo with green-tinted digital color grading	Green-tinted digital aesthetic
Grand Budapest	Photo in pastel, symmetrical Wes Anderson style	Wes Anderson pastel symmetry

Fashion section (1x3 row):

Panel	Image	Description
Vogue Editorial	Photo in high-fashion editorial studio lighting	High-fashion studio lighting
90s Streetwear	Photo with retro grain and muted 90s tones	Retro grain and muted tones

Panel	Image	Description
Cyber-Tech	Photo in futuristic neon tech aesthetic	Futuristic neon tech aesthetic

Active: border glow matching section color (golden for Lighting, cyan for Style).

HEXCodes tab — Color Palette Extraction & Presets:

```
┌──────────────────────────────────────────┐
│ 📷 CAMERA  │ 💡 LIGHTING  │ 🎨 STYLE  │ 🎨 HEXCODES  │
│            │              │           │  ═══════════  │
├──────────────────────────────────────────┤
│                                          │
│  EXTRACT FROM IMAGE                      │
│  ┌──────────────────────────────────┐    │
│  │                                  │    │
│  │     📷  Drop image or click to   │    │
│  │         upload for extraction    │    │
│  │                                  │    │
│  └──────────────────────────────────┘    │
│                                          │
│  ── after upload ──                      │
│                                          │
│  EXTRACTED COLORS                        │
│  ┌───────┐ ┌───────┐ ┌───────┐ ┌───────┐        │
│  │▇▇▇▇▇▇▇│ │▇▇▇▇▇▇▇│ │▇▇▇▇▇▇▇│ │▇▇▇▇▇▇▇│  [+]  │
│  │#2A1B3D│ │#E94560│ │#0F3460│ │#533483│       │
│  └───────┘ └───────┘ └───────┘ └───────┘        │
│  [✓ Use]                    [↻ Re-extract]  │
│                                          │
├──────────────────────────────────────────┤
│                                          │
│  PREMADE PALETTES                        │
│                                          │
│  ┌──────────────────────────────────┐    │
│  │ ▇▇▇▇  Matrix Green               │    │
│  │ #003B00  #00FF41  #0D0208  #008F11  │    │
│  └──────────────────────────────────┘    │
│  ┌──────────────────────────────────┐    │
│  │ ▇▇▇▇  Cyberpunk Neon             │    │
│  │ #FF006E  #8338EC  #3A86FF  #FB5607  │    │
│  └──────────────────────────────────┘    │
│  ┌──────────────────────────────────┐    │
│  │ ▇▇▇▇  Desert Warmth              │    │
│  │ #E07A5F  #F2CC8F  #81B29A  #3D405B  │    │
│  └──────────────────────────────────┘    │
│  ┌──────────────────────────────────┐    │
│  │ ▇▇▇▇  Noir                       │    │
│  │ #1A1A2E  #16213E  #0F3460  #E94560  │    │
│  └──────────────────────────────────┘    │
│  ┌──────────────────────────────────┐    │
│  │ ▇▇▇▇  Golden Hour                │    │
│  │ #FF9A3C  #FF6F00  #E65100  #FFD54F  │    │
│  └──────────────────────────────────┘    │
│                                          │
├──────────────────────────────────────────┤
│                                          │
```

```
│  CUSTOM SWATCHES                              │
│  ┌──────┐  ┌──────┐  ┌──────┐  ┌──────┐      │
│  │######│  │######│  │######│  │  +   │      │
│  │#FF5733│ │#1DB954│ │#6C5CE7│ │ add  │      │
│  └──────┘  └──────┘  └──────┘  └──────┘      │
│                                               │
│  Click swatch to edit hex · Click + to add    │
│  Long press to delete                         │
│                                               │
├───────────────────────────────────────────────┤
│  ACTIVE PALETTE                               │
│  ■ #2A1B3D  ■ #E94560  ■ #0F3460              │
│                    [Reset]  [Apply ▸]         │
└───────────────────────────────────────────────┘
```

Section details:

Section	Description
Extract from Image	Drag & drop or click to upload an image. Primary hex colors are auto-extracted using dominant color analysis. Displays 4–6 extracted swatches with hex codes. Click "Use" to apply as active palette, or "Re-extract" to try again.
Premade Palettes	Scrollable list of predefined cinematic palettes. Each row shows color preview squares, palette name, and 4 hex codes. Click a palette to select it. Active palette: orange border. Palettes: Matrix Green, Cyberpunk Neon, Desert Warmth, Noir, Golden Hour, Pastel Dream, Blade Runner, Wes Anderson.
Custom Swatches	User-created hex swatches. Click a swatch to open a hex input/color picker to edit. Click "+" to add a new swatch. Long press a swatch to delete. Max 8 custom swatches.
Active Palette (bottom bar)	Shows the currently selected hex colors as small squares with codes. These colors are appended to the prompt tags when Cinematographer is ON.

Summary bar at bottom shows all selected options. **[Reset]** clears to defaults. **[Apply]** saves and closes modal.

Database storage: All selected options saved as `cinematographer_settings` JSON on the ShotFile when generating.

7g. Parameters (2x2 grid)

Top row (always visible):

Parameter	Type	Options	Default
Aspect	dropdown	16:9 Cinema, 4:3, 1:1, 3:2, 9:16 (varies by model)	16:9 Cinema
Resolution	dropdown	model-specific (e.g., 720p, 1080p, 4K)	model default

Bottom row (shown per model — only when the model exposes these settings):

Parameter	Type	Options	Default
CFG Scale	number input	Guidance scale. Typically 1–20.	7.5
Seed	number input	"Auto" by default (random). Set a specific number for reproducibility.	Auto

The bottom row renders dynamically based on the selected model's available parameters. Some models may show Steps instead of CFG Scale, or additional model-specific controls.

7h. Advanced Settings (collapsible)

- Additional model-specific parameters.
- Same collapsible chevron pattern as elsewhere.

7i. Generate Button (sticky bottom bar)

- Sticky bar at the very bottom of the controls pane.
- **Left:** Grid icon button (opens batch/grid generation options).
- **Center:** Full-width pill button, orange gradient (`from-orange-600 to-amber-400`). White text: "GENERATE".
- **Right:** Batch count dropdown "1 ▾" — select how many images to generate per click (1, 2, 4).
- Hover: scale 1.05, intensified glow shadow.
- Active/pressed: scale 0.95.
- Not disabled during generation — shows a spinner/processing state while inputs are being prepared, then returns to clickable once the job is dispatched.
- **Keyboard shortcut:** ⌘+Enter (Mac) / Ctrl+Enter.

Feature 8: Generation Controls — Video Tab

What it does: Controls for video generation. Shares the same controls pane column but with video-specific inputs.

Reference: `src/components/ControlsPane.tsx` lines 3582-4300

Layout (top to bottom):

```
┌──────────────────────────────────────┐
│ [Cinematic Video Engine v2.4     ▼]  │  ← Model selector
├──────────────────────────────────────┤
│ KEYFRAMES                            │
│  ┌─────────┐  ┌─────────┐            │
│  │   📷    │  │   📷    │   [▸]      │
│  │ START   │  │ END     │            │
│  │ FRAME   │  │ FRAME   │            │
│  └─────────┘  └─────────┘            │
├──────────────────────────────────────┤
│ PROMPT                               │
│  ┌──────────────────────────────┐    │
│  │ > A fast-tracking side shot of │  │
│  │   a motorcycle weaving through │  │
│  │   neon traffic, rain droplets  │  │
│  │   hitting the camera lens...   │  │
│  │ [↻][~][🗑]              [✨  ▾] │  │
│  └──────────────────────────────┘    │
├──────────────────────────────────────┤
│ DURATION                     04.0s   │
│ [═══════════●──────────────────]     │  ← Slider
├──────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐  │
│  │ RESOLUTION   │  │ AUDIO        │  │
│  │ 8K UHD     ▼ │  │ ATMOS    [🔘] │  │
│  └──────────────┘  └──────────────┘  │
│  ┌──────────────┐  ┌──────────────┐  │
│  │ MOTION       │  │ FRAMERATE    │  │
│  │ Dynamic (7)  │  │ 24 Cinematic │  │
│  └──────────────┘  └──────────────┘  │
├──────────────────────────────────────┤
│ Advanced Export Settings           ▸ │  ← Collapsible
├──────────────────────────────────────┤
│                                      │
│   [▓▓▓▓ GENERATE (⚡ 4.0) ▓▓▓▓]      │  ← Sticky footer
└──────────────────────────────────────┘
```

8a. Model Selector

- Same style as image tab. Groups models under "Video Pipelines" optgroup.
- **Tab memory:** Remembers last selected model when switching tabs.

8b. Keyframes & References

The keyframes section has **two modes**, toggled by the ▾ **down arrow** on the right:

Default mode (Keyframes only):

```
┌──────────────────────────────────┐
│ KEYFRAMES                        │
│ ┌─────────┐  ┌─────────┐         │
│ │   📷    │  │   📷    │   [▸]   │
│ │ START   │  │ END     │         │
│ │ FRAME   │  │ FRAME   │         │
│ └─────────┘  └─────────┘         │
└──────────────────────────────────┘
```

Reference mode (expanded via ▾ arrow):

```
┌──────────────────────────────────┐
│ KEYFRAMES & REFERENCES        [▴] │
│ ┌──────┐ ┌──────┐ ┌──────┐ [+]   │
│ │ REF  │ │ ELF  │ │  📷  │       │
│ │ img  │ │ img  │ │      │   [▾] │
│ └──────┘ └──────┘ └──────┘       │
└──────────────────────────────────┘
```

- Clicking the ▾ **down arrow** collapses the full-size keyframe zones and switches to a compact horizontal row of thumbnails.
- The **first two thumbnails** are always the start and end frames, marked with **SF** (start frame) and **EF** (end frame) corner badges respectively.
- Each thumbnail has a **color-coded corner badge** identifying its type:
  - **SF** (amber badge) — start frame
  - **EF** (amber badge) — end frame
  - **REF** (orange badge) — reference image
  - **ELF** (cyan badge) — element character sheet
- Thumbnails are ~48x48px rounded squares in a single row. **Drag to reorder** references and elements.
- **[+] button** to add more references, elements, or keyframes.
- ▾ **down arrow** on the row toggles back to the full keyframe view.
- Elements added here use the same character sheet approach as the image tab — injected as reference images, not model-specific APIs.

8c. Prompt Area

- Same textarea as image tab with identical toolbar layout:
  - Bottom-left: Undo (↻), Redo (↷), Clear (🗑)
  - Bottom-right: ✨ Auto-Expand with ▾ dropdown templates

8d. Multi-Shot Sequence Mode

When SEQUENCE mode is active (replaces the single prompt with a shot list):

```
┌──────────────────────────────────────┐
│ SEQUENCE                             │
│  ┌──────────────────────────────────┐ │
│  │ SHOT 01                   5.0s   │ │  ← Collapsed (click to expand)
│  └──────────────────────────────────┘ │
│  ┌──────────────────────────────────┐ │
│  │ SHOT 02                   3.5s   │ │  ← Active (expanded)
│  │ > Close-up of a cybernetic eye   │ │
│  │   reflecting neon city lights,   │ │
│  │   high detail, micro-...         │ │
│  │ [↻][~][🗑]              [✨ ▾]   │ │
│  └──────────────────────────────────┘ │
│  ┌──────────────────────────────────┐ │
│  │ SHOT 03                   3.5s   │ │  ← Collapsed
│  └──────────────────────────────────┘ │
│  ┌──────────────────────────────────┐ │
│  │               [+]                │ │  ← Add shot
│  └──────────────────────────────────┘ │
└──────────────────────────────────────┘
```

- Each shot row shows: **SHOT ##** label with colored left-edge accent, duration on the right.
- Click a collapsed shot to expand it and edit its prompt inline (same textarea toolbar as single-prompt mode).
- **Duration** per shot shown on the right side of each row header (e.g., "5.0s", "3.5s").
- **[+] button** at the bottom adds a new shot to the sequence.
- All shots share the same keyframes, references, elements, and parameters.
- ⌘+Enter generates the full multi-shot sequence.

8e. Duration

- **Horizontal slider** with orange track and circular thumb.
- Duration value shown top-right in orange text (e.g., "04.0s").
- Range varies by model (typically 2s–15s). Step: 0.5s.
- In sequence mode, this is replaced by per-shot durations in each shot row.

8f. Video Parameters (2x2 grid)

Parameter	Type	Options
Resolution	dropdown	720p, 1080p, 4K UHD, 8K UHD (varies by model)
Audio	labeled toggle	ATMOS on/off. Orange toggle when active. Enables spatial audio generation alongside video.
Motion	stepper/dropdown	Motion intensity level. e.g., "Dynamic (7)". Range 1–10.
Framerate	dropdown	24 Cinematic, 30 Standard, 60 Smooth

8g. Advanced Export Settings (collapsible)

- Contains model-specific advanced parameters (guidance, seed, codec options, etc.).
- Same collapsible pattern as image tab's Advanced Settings.

Feature 9: Generation Controls — Other Tabs

⚠️ **NOT IN SPRINT 1 SCOPE:** All tabs other than Image and Video are deferred to later sprints. These are documented here for context only, no implementation required for Sprint 1.

9a. Tools Tab

- Same layout as Image/Video but models are "Special Pipelines":
  - Video-to-Video (style transfer)
  - Image Upscale
  - Frame Interpolation
  - Background Removal
- Input is typically a file (image or video) rather than a text prompt.
- Parameters are model-specific and rendered dynamically.

9b. Audio Tab

```
┌──────────────────────────────────┐
│    🎵                            │
│  Audio Generation                │
│  Sound effects, music, and       │
│  voice synthesis coming soon.    │
│                                  │
│  ┌────────────────────────┐      │  ← skeleton wireframes
│  │                        │      │
│  └────────────────────────┘      │
│  ┌────────────────────────┐      │
│  │                        │      │
│  └────────────────────────┘      │
│  ┌────────────────────────┐      │
│  │                        │      │
│  └────────────────────────┘      │
└──────────────────────────────────┘
```

- Static placeholder with icon, title, subtitle, and 3 skeleton boxes.
- No functionality. Ships fully in Sprint 2.

9c. Chat Tab (AI Prompt Assistant)

```
┌──────────────────────────────────┐
│ [✦ Prompt Engineer ▼]            │  ← system prompt selector
├──────────────────────────────────┤
│                                  │
│        Write me a cinematic   [U]│  ← user message (right-aligned)
│        prompt for a scarecrow    │
│        walking through mist...   │
│                                  │
│  [AI] Here's a cinematic prompt: │  ← AI response (left-aligned)
│                                  │
│  ┌─ Generated Prompt ─── [Copy]┐│  ← code block
│  │ A weathered scarecrow with   │
│  │ tattered burlap clothing     │
│  │ slowly walks through endless │
│  │ rows of golden corn...       │
│  └──────────────────────────────┘│
│                                  │
│  [AI] I focused on volumetric   │
│  lighting and film texture...    │
│                                  │
├──────────────────────────────────┤
│ [Ask anything...         ] [▶]  │  ← input + send
└──────────────────────────────────┘
```

System prompt presets dropdown:

Preset	Behavior
General Assistant	Broad help with any question
Creative Director	Focuses on composition, storytelling, mood
Shot Breakdown	Decomposes a scene description into individual shots
Prompt Engineer	Optimizes prompts for specific AI models
Storyboard Advisor	Suggests shot sequences and transitions

Message layout:

- User messages: right-aligned, amber-tinted background, "U" avatar.
- AI messages: left-aligned, subtle white background, "AI" avatar (violet).
- Code blocks: dark background, monospace font, "Generated Prompt" header, **[Copy]** and **[Inject]** buttons.
  - **[Copy]**: copies prompt text to clipboard.
  - **[Inject]**: currently disabled (no-op) to avoid UI bugs. In Sprint 2: inserts the prompt into the active generation tab's prompt textarea.

Input area:

- Textarea (1 row, auto-grows), "Ask anything..." placeholder.
- Orange gradient send button [▶] next to it.

Backend: `POST /api/chat/` — proxies to an LLM API with the selected system prompt prepended.

Feature 10: Generation Flow (End to End)

What happens when the artist clicks "Generate":

```
1. FRONTEND: Validates inputs
   - Prompt not empty (unless model doesn't require it)
   - Required references provided
   - Model selected
   - Parameters within valid ranges

2. FRONTEND: Collects ALL parameters into a request:
   {
     shot_id,
     model,
     prompt,
     prompt_tags: [...auto-extracted keywords],
     cinematographer_settings: { camera_body, lens, focal, fstop, lighting, style },
     model_settings: { aspect_ratio, duration, resolution, seed, guidance, ... },
     ref_file_ids: [ids of uploaded reference files],
     element_ids: [ids of selected elements],
     multishot_prompts: [{prompt, duration}, ...] (if multishot mode)
   }

3. FRONTEND: POST /api/generate/

4. BACKEND: Creates GenerationJob record (status: "pending")
   - Stores ALL settings on the job record for full reproducibility

5. BACKEND: Dispatches to background worker queue

6. BACKEND: Returns job_id immediately

7. FRONTEND: Shows "processing" tile in the shot grid
   - Animated gradient shimmer background
   - "PROCESSING FRAME..." or "RENDERING VIDEO..." center text
   - Progress bar at bottom

8. FRONTEND: Polls GET /api/generate/{job_id}/ every 2 seconds
   - Updates progress bar based on response.progress (0-100)

   --- Meanwhile, in the background worker ---
   a. Resolves ref File records → downloads from Object Storage if needed
   b. Resolves element File records → downloads from Object Storage if needed
   c. Builds API request for the AI model provider
   d. If cinematographer settings are active, appends camera text to prompt
   e. Calls the AI model API (Kling, Flux, etc.)
   f. Waits for completion (some providers are async themselves)
   g. Downloads the result
   h. Generates filename: {timestamp}_{user}_{model}_{hash}.{ext}
   i. Uploads to Object Storage: {project_id}/generations/{filename}
   j. Creates File record with ALL metadata (prompt, settings, elements)
   k. Creates FileAssignment(file=new_file, shot=target_shot, role="output")
   l. Sets refs_used M2M on the File to link input refs
```

```
    m. Updates job status to "completed" with the file_id

 9. FRONTEND: Poll returns status "completed" with file metadata

10. FRONTEND: Replaces processing tile with the actual output tile

11. FRONTEND: Records prompt to history (POST /api/prompts/)
```

Shot-scoped generation: The `shot_id` is captured at job creation time. If the artist switches to a different shot while generation is running, the output still lands in the original shot.

Retry flow:
When the user clicks the **Retry** button on an output tile:

1. Frontend reads the original job's parameters from the ShotFile metadata
2. Populates generation controls with exact same parameters
3. Backend creates a new GenerationJob with identical settings but a new random seed
4. Same flow as above continues

Feature 11: Project Assets (Characters, Locations, Props)

What it does: Shared project-level resources uploaded once and available as references across all shots.

```
┌──────────────────────────────────────────┐
│  Asset Store                          [X]  │
│                                            │
│  [Characters] [Locations] [Props] [Shots]  │
│                                            │
│  ┌─────────┐  ┌─────────┐  ┌─────────┐    │
│  │scarecrow│  │ farmer  │  │  crow   │    │
│  │  main   │  │  wife   │  │         │    │
│  └─────────┘  └─────────┘  └─────────┘    │
│  ┌─────────┐                               │
│  │  + Add  │                               │
│  │  Asset  │                               │
│  └─────────┘                               │
│                                            │
│  Drag an asset into a shot to use as ref   │
└──────────────────────────────────────────┘
```

Tabs:

- **Characters** — character sheets, turnarounds, faces
- **Locations** — environment references
- **Props** — object references

- **Shots** (Sprint 2) — browse outputs from other shots to use as refs

When you drag an asset into a shot or click "Use as Reference", the backend creates a `FileAssignment(file=asset_file, shot=target_shot, role="input")`. No file is copied — the asset's File record is simply tagged as an input for that shot.

Feature 12: Elements Manager

What it does: Create and manage structured character/object definitions used for face/character consistency in models like Kling 3.0.

Different from Project Assets: Assets are loose image files. Elements are structured entities with a primary frontal image, up to 3 reference angles, a name, and a description.

```
┌──────────────────────────────────────────────┐
│  Elements                              [X]    │
│                                               │
│  ┌─────────┐  ┌─────────┐  ┌──────────┐      │
│  │ frontal │  │ frontal │  │  + New   │      │
│  │ (face)  │  │ (face)  │  │ Element  │      │
│  │─────────│  │─────────│  │          │      │
│  │Scarecrow│  │ Farmer  │  │ (dashed) │      │
│  │ Main    │  │  Wife   │  │          │      │
│  └─────────┘  └─────────┘  └──────────┘      │
│                                               │
│  Click an element to view/edit:               │
│  ┌──────────────────────────────────────┐     │
│  │ Name: [Scarecrow Main        ]       │     │
│  │ Description: [Tattered burlap...]    │     │
│  │                                      │     │
│  │ Frontal: ┌────┐  Refs: ┌──┐┌──┐┌──┐│     │
│  │          │face│        │45°│90°│bk│     │
│  │          └────┘        └──┘└──┘└──┘│     │
│  │                          [+ Add Ref]│     │
│  └──────────────────────────────────────┘     │
└──────────────────────────────────────────────┘
```

Creating an element:

1. Upload a frontal image (required) — the primary face/body reference
2. Optionally add up to 3 reference images (45-degree angle, side profile, back, etc.)
3. Name the element (e.g. "Scarecrow Main", "Hero Car")
4. Add optional description

Using an element:

1. In the Image or Video tab controls, click "+ Add from Elements"
2. Elements Manager opens in selection mode
3. Click an element to add it to the generation
4. Reference it in the prompt with `@Element1`

Backend:

- Elements stored in `{project_id}/elements/{element_id}/`
- `POST /api/projects/{id}/elements/` — multipart upload with frontal + refs
- Model-specific: When generating with Kling 3.0, the backend uploads element images to Kling's API and maps `@Element1` tags to the uploaded URLs

Feature 13: Tag as Input Ref from Another Shot

What it does: Artist working on SH05 uses an output from SH02 as a reference — without copying.

Flow:

1. Artist scrolls to SH02 (inactive shot) in the grid
2. Hovers on the desired output tile
3. Clicks the 📎 **Use as Reference** button (bottom-left hover controls)
4. Frontend calls `POST /api/shots/{sh05_id}/assignments/` with `{ file_id, role: "input" }`
5. Backend creates a FileAssignment: `(file=file_xyz, shot=sh05, role="input")`
6. **No file is copied on Object Storage.** The file stays in its original location.
7. Frontend refreshes SH05's file list — the tagged file appears with an emerald border

Why this is better:

- Zero Object Storage storage cost for ref usage
- If the artist later removes the ref tag, the original file is untouched
- You can query "where is this file used?" by listing its assignments
- If the file is soft-deleted, it vanishes from all shots simultaneously (and can be restored to bring it back everywhere)

Same file, multiple tags example:

```
File: "scarecrow_golden_hour.mp4" (generated in SH02)

Assignments:
  → output of SH02        (where it was generated)
  → input ref of SH05     (artist is using it as ref)
  → input ref of SH08     (another artist reused it)
  → pinned (global)       (artist pinned for quick access)
```

Feature 14: Publish/Star a Candidate

What it does: Mark an output as the final pick for a shot.

Flow:

1. Artist hovers on an output tile, clicks the ⋮ **More menu** → "Set as Published"
2. Frontend calls `POST /api/shots/{shot_id}/publish/` with the file ID
3. Backend:
   - Sets `published=true` on the ShotFile
   - Sets `published=false` on any previously published file
   - Updates `shot.published_output` to this file's Object Storage key
4. Frontend shows ★ published badge on the tile (violet circle, top-left)

In Sprint 1 this is a local marker. In Sprint 2, published outputs get pinned rightmost on the global board and optionally pushed to Kitsu.

Django Project Structure

```
ark_backend/
├── manage.py
├── requirements.txt
├── ark/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py           # Sprint 2: switch to asgi.py for Channels
├── projects/
│   ├── models.py          # Project, Scene, Shot, ShotFile, ProjectAsset, Element
│   ├── serializers.py
│   ├── views.py           # REST views for all CRUD
│   ├── urls.py
│   └── services/
│         ├── s3.py         # Object Storage upload, download, copy, presigned URLs
│         └── manifest.py   # Read/write manifest.json
├── generation/
│   ├── models.py          # GenerationJob, PromptHistory
│   ├── serializers.py
│   ├── views.py           # Generate, poll, retry endpoints
│   ├── urls.py
│   └── tasks.py           # Background async generation tasks
├── chat/
│   ├── views.py           # LLM proxy endpoint
│   └── prompts.py         # System prompt presets
└── auth/
    ├── models.py          # User
    ├── views.py           # Login, register, refresh
    └── urls.py
```

Next.js Project Structure

```
src/
├── app/
│   ├── page.tsx                    # Router: home vs workspace
│   ├── layout.tsx
│   └── globals.css
├── components/
│   ├── HomeScreen.tsx              # Project list + create
│   ├── CreateProjectModal.tsx      # New project form
│   ├── FileBrowser.tsx             # Shot grid + toolbar
│   ├── ShotContainer.tsx           # Single shot with candidates
│   ├── CandidateTile.tsx           # Individual file tile with hover controls
│   ├── ControlsPane.tsx            # Generation controls with vertical tabs
│   ├── ImageShotLookCards.tsx      # Cinematographer summary + modal trigger
│   ├── CameraMovementSelector.tsx  # Camera movement popover
│   ├── ImageEditor.tsx             # Fullscreen image viewer/editor
│   ├── VideoPlayer.tsx             # Fullscreen video player
│   ├── AssetStore.tsx              # Project assets modal
│   ├── ElementsManager.tsx         # Elements create/manage/select panel
│   └── ui/
│         └── Spinner.tsx
├── state/
│   ├── auth.tsx                    # Auth context + JWT management
│   ├── project.tsx                 # Active project context
│   ├── shots.tsx                   # Shots for active scene
│   ├── queue.tsx                   # Generation job queue + polling
│   └── elements.tsx                # Elements selection state
├── lib/
│   ├── api.ts                      # Base fetch wrapper with auth headers
│   ├── projects.ts                 # Project API calls
│   ├── shots.ts                    # Shot + file API calls
│   ├── generation.ts               # Generation API calls
│   ├── chat.ts                     # Chat API calls
│   ├── models.ts                   # Model specs, params, capabilities
│   ├── image-models.ts             # Image model definitions
│   ├── special-models.ts           # Special/tool model definitions
│   ├── providers.ts                # AI provider API clients
│   ├── prompt-builder/             # Cinematographer prompt construction
│   │     └── types.ts              # ShotSettings, LookSettings
│   └── llm.ts                      # Prompt expansion/alteration via LLM
└── types/
    └── index.ts                    # Shared TypeScript types
```

Sprint 2 Readiness

Everything in Sprint 1 is built to accommodate Sprint 2 without breaking changes:

Sprint 2 Feature	How Sprint 1 Prepares
Global board	`outputs/` folder convention + board API endpoint already designed. ShotFile has `user` field ready for multi-user attribution.
Multi-user	`user` FK on ShotFile, GenerationJob, PromptHistory. Auth system ready.
WebSockets	Sprint 1 uses polling with `setInterval`. Sprint 2 replaces with WS but keeps polling as fallback. `queue.tsx` context provider abstracts this.
Kitsu integration	`kitsu_id` fields commented in models — just uncomment and add migration. "Import from Kitsu" button already in CreateProjectModal (grayed out).
Comments	ShotFile has a stable UUID PK for a future `FileComment` FK.
Audio generation	Audio tab placeholder exists. GenerationJob model is format-agnostic.
Canvas view	ShotFile stores enough metadata (thumbnail, shot order) for canvas cards.

Environment Variables

Backend

```
SECRET_KEY=django-secret-key
DATABASE_URL=postgres://...
# No external queue service required in Sprint 1
OBJECT_STORAGE_ACCESS_KEY=...
OBJECT_STORAGE_SECRET_KEY=...
OBJECT_STORAGE_BUCKET=ark-production
OBJECT_STORAGE_REGION=...

# Provider agnostic object storage.
# Works with S3, GCS, Cloudflare R2, MinIO, or any S3-compatible service.
CORS_ALLOWED_ORIGINS=http://localhost:3000
```

Frontend

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api
```